

Question 1:

An $N \times N$ matrix representation of a graph is given as input, where 1 at Mth row and Nth column denotes node M and node N are connected directly. A 0 denotes the two nodes are disconnected. Perform a depth-first traversal and print the nodes in the traversing order. When visiting the neighboring node, the priority must be given to the node with least value to visit first.

Boundary Condition(s):

$1 \leq N \leq 100$

Input Format:

The first line contains the value of N and source node separated by space(s).

The next N lines contain N elements each separated by space(s).

Output Format:

The first line contains nodes separated by a space.

Example Input/Output 1:

Input:

```
5 1
1 1 1 0 1
1 1 0 1 0
1 0 1 1 1
0 1 1 1 1
1 0 1 1 1
```

Output:

```
1 2 4 3 5
```

Example Input/Output 2:

Input:

```
4 1
1 0 1 0
0 1 1 1
1 1 1 0
0 1 1 1
```

Output:

```
1 3 2 4
```

ALGORITHM:

1. Input N, source node
2. Initialize adjacency matrix adjMatrix of size $(N+1) \times (N+1)$
3. Initialize visited array of size $(N+1)$ to 0
4. Input the adjacency matrix values from the user
5. Call depth_first(adjMatrix, source, visited)

Procedure depth_first(adjMatrix, curr, visited):

1. Print the current node curr
2. Mark visited[curr] = 1 (node is now visited)
3. For each col from 1 to N:
4. If adjMatrix[curr][col] == 1 and visited[col] == 0:
5. Call depth_first(adjMatrix, col, visited)
6. After DFS traversal:
 1. Free the memory allocated for adjMatrix
 2. Free the memory allocated for visited

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

int N;

void depth_first(int **adjMatrix, int curr, int *visited) {
    printf("%d ", curr);
    visited[curr] = 1;
    for (int col = 1; col <= N; col++) {
        if (adjMatrix[curr][col] == 1 && visited[col] == 0) {
            depth_first(adjMatrix, col, visited);
        }
    }
}

int main() {
    int row, col, source;
    scanf("%d %d", &N, &source);
    int **adjMatrix = (int **)malloc((N + 1) * sizeof(int *));
    for (row = 0; row <= N; row++) {
```

```

adjMatrix[row] = (int *)malloc((N + 1) * sizeof(int));
}
for (row = 1; row <= N; row++) {
    for (col = 1; col <= N; col++) {
        scanf("%d", &adjMatrix[row][col]);
    }
}

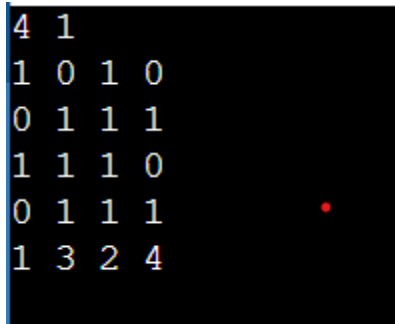
int *visited = (int *)malloc((N + 1) * sizeof(int));
memset(visited, 0, (N + 1) * sizeof(int));
depth_first(adjMatrix, source, visited);

for (row = 0; row <= N; row++) {
    free(adjMatrix[row]);
}

free(adjMatrix);
free(visited);
return 0;
}

```

OUTPUT



```

4 1
1 0 1 0
0 1 1 1
1 1 1 0
0 1 1 1
1 3 2 4

```

Question 2:

The program must accept N integers as input. The program must append an integer to the list if the integer is even. Else the program must prepend the integer to the list. Finally, the program must print the list. Fill in the missing lines of code by using append and prepend function given in the header file so that the program runs successfully.

Example Input/Output 1:

Input:

5
4 5 1 2 3

Output:

3 1 5 4 2

Explanation:

1st integer is 4 (even) append 4 to the list -> The list contains 4
2nd integer is 5 (odd) prepend 5 to the list -> The list contains 5 4
3rd integer is 1 (odd) prepend 1 to the list -> The list contains 1 5 4
4th integer is 2 (even) append 2 to the list -> The list contains 1 5 4 2
5th integer is 3 (odd) prepend 3 to the list -> The list contains 3 1 5 4 2
Hence the output is 3 1 5 4 2

Algorithm:

1. Initialization:

- Create an integer array array of size MAX_SIZE (100).
- Set size = 0 to track the number of elements in the array.

2. Function Definitions:

- append(int val):
 - If size < MAX_SIZE (there's space in the array), insert the given value val at the current size index and then increment the size (size++).
- prepend(int val):
 - If size < MAX_SIZE (there's space in the array), shift all elements in the array one position to the right, starting from the last element down to the first. Then, insert the value val at the 0th position and increment the size (size++).
- getSize():
 - Return the current size of the array.

3. Main Function:

- Read the number n, which represents how many elements will be input.

- Read n integers, one at a time:
 - For each integer:
 - If the integer is even ($\text{val} \% 2 == 0$), call the append function to add the integer at the end of the array.
 - If the integer is odd ($\text{val} \% 2 != 0$), call the prepend function to add the integer at the beginning of the array.
- After processing all n integers, print the array elements in order from the 0th index to size - 1.

-

CODE:

```
#include <stdio.h>

#define MAX_SIZE 100

int array[MAX_SIZE];

int size = 0;

void append(int val) {
    if (size < MAX_SIZE) {
        array[size++] = val; // Add the value and increment size
    }
}

void prepend(int val) {
    if (size < MAX_SIZE) {
        for (int i = size; i > 0; i--) {
            array[i] = array[i - 1];
        }
        array[0] = val;
        size++;
    }
}

int getSize() {
    return size;
}
```

```

}

int main() {
    int n, val;

    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the values: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        if (val % 2 == 0) {
            append(val);
        } else {
            prepend(val);
        }
    }

    printf("Result: ");
    for (int index = 0; index < getSize(); index++) {
        printf("%d ", array[index]);
    }

    printf("\n");
    return 0;
}

```

OUTPUT:

```

enter the number:5
enter the values:4 5 6 7 8
Result:7 5 4 6 8

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 3:

An array of N positive integers are passed as input. The program must form a binary search tree with these numbers. The first number (out of the N numbers passed as input) is the root node of the binary search tree. Implement the function **insert()** so that the program can print the pre-order traversal of the tree formed (calling the pre-existing function supplied in the header file **preorder.h**)

The definition of the **struture Node** is

```
struct Node
{
    int data;
    struct Node *left, *right;
};
```

Example Input/Output 1:

Input:

6
3 2 4 1 7 6

Output:

3 2 1 4 7 6

Algorithm:

1. Initialization:

- Create an integer array array of size MAX_SIZE (100).
- Set size = 0 to track the number of elements in the array.

2. Function Definitions:

- append(int val):
 - If size < MAX_SIZE (there's space in the array), insert the given value val at the current size index and then increment the size (size++).
- prepend(int val):
 - If size < MAX_SIZE (there's space in the array), shift all elements in the array one position to the right, starting from the last element down to the first. Then, insert the value val at the 0th position and increment the size (size++).
- getSize():
 - Return the current size of the array.

3. Main Function:

- Read the number n, which represents how many elements will be input.
- Read n integers, one at a time:

- For each integer:
 - If the integer is even ($\text{val} \% 2 == 0$), call the append function to add the integer at the end of the array.
 - If the integer is odd ($\text{val} \% 2 != 0$), call the prepend function to add the integer at the beginning of the array.
- After processing all n integers, print the array elements in order from the 0th index to size - 1.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *left, *right;
};
```

```
struct Node *root = NULL;
```

```
struct Node* newNode(int data) {
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}
```

```
struct Node* insert(struct Node* root, int data) {
    if (root == NULL) {
```



```

        return newNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    }

    else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

void printPreorder(struct Node *node) {
    if (node != NULL) {
        printf("%d ", node->data);
        printPreorder(node->left);
        printPreorder(node->right);
    }
}

int main() {
    int N, data, ctr;

    scanf("%d", &N);

    for (ctr = 1; ctr <= N; ctr++) {
        scanf("%d", &data);
    }
}

```

```
        root = insert(root, data);  
    }  
    printPreorder(root);  
    printf("\n");  
    return 0;  
}
```

OUTPUT:

```
5  
12 8 9 10 5  
12 8 5 9 10  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```